

New Dispo: Projektscope und Abgrenzung zu TMS Core

Management Summary

Datum: 12.12.2025

Projektziel: New Dispo ist eine Cloud-Applikation, die den TMS Core um moderne Dispositionsfunktionen erweitert, ohne dessen operative Bestandslogiken zu ersetzen.

Architektur-Ansatz: New Dispo steuert TMS Core "fern" und nutzt bestehende TMS Datenbank-Views, Packages und den TOP-Service/xServer für Tourenberechnung. Architektur-Entscheidungen sind im Decision Log, ADRs und Refinement Contexts dokumentiert.

Infrastruktur: Der New Dispo Stack (Frontend, Backend, TMS Bridge, TMS Pulse) läuft in GCP; TOP-Service und xServer sind On-Premise. Die Netzwerkverbindung Nagel ↔ GCP sowie die Anbindung von On-Premise Komponenten ↔ GCP liegt außerhalb des P3-Verantwortungsbereichs.

Zentrale Herausforderungen:

- **Performance-Bottleneck im TMS Core:** Verschachtelte Datenbank-Funktionen führen zu langsamen Datenzugriffen, die auch durch neue Views und Funktionen nicht vollständig behoben werden können
- **Project G Migration:** PostgreSQL-Performance teilweise schlechter als Oracle; offene Probleme bestehen bis heute
- **Abhängigkeiten:** P3 hat keinen Einfluss auf die drei kritischen Komponenten TMS Core DB-Schnittstelle, TOP-Service und xServer (alle in CAL-Verantwortung)

Erstellte Datenbank-Objekte: Um den initialen Bottlenecks bei den bereitgestellten Objekten zu begegnen, hat P3 zahlreiche Use-Case-spezifische Objekte erstellt oder weiterentwickelt: Packages (PDIS_) und Views (V_DIS_) (siehe Tabelle unten) – allerdings nicht überall mit 100% Erfolg.** Um alle Performance-Herausforderungen grundsätzlich zu beheben, ist eine tiefere Analyse und Neugestaltung des TMS Core notwendig, was nicht Teil des Projektsopes von New Dispo war.**

Fazit: P3 hat auf die infrastrukturellen Performance-Probleme zwischen Nagel-Netzwerk und GCP keinen Einfluss. Zudem hat P3 alles im Rahmen des Möglichen getan, um die Use Cases von New Dispo mit der bestmöglichen Performance abzubilden – sogar insoweit, als dass zahlreiche verschiedene TMS Core Objekte erstellt wurden. Allerdings liegt das Potenzial, die restlichen bestehenden Probleme zu beheben, weit tiefer im TMS Core und erfordert einen dedizierten Ansatz bzw. ein separates Projekt.

Initiale Architektur-Philosophie

- Project G bringt alle Oracle Instanzen in die Cloud als schnelle, performante Basis für zukünftige Dispositionsanwendungen
- CAL stellt Schnittstellen für alle Pickup-Planning relevanten Entitäten bereit (Shipments, Transport Orders, Tourpunkte)
- Sinnbild: New Dispo steuert TMS Core "fern" wo Bestandslogiken den operativen Betrieb sicherstellen müssen und erweitert den Core um neue Entitäten und Features, die der TMS Core nicht unterstützt, welche aber auf den Daten, Funktionen und Schnittstellen des Core aufbauen
- Schnelle Umsetzung sollte durch Nutzung von soviel wie möglich bestehendem TMS-Stack sichergestellt werden
 - Bereitgestellte Basis-Datenbank-Views u.a.
 - V_DIS_TRANSPORTORDER
 - V_DIS_SHIPMENT
 - V_DIS_T0_TOURPOINT
 - Bereitgestellte Basis-Funktionen & Schemas u.a.
 - pDis_TransportOrderDT0.Get()
 - PDIS_TRANSPORTORDER
 - TOP-Service und xServer für Tourenberechnung (On-Premise gehostet)
 - Bestehende REST-Schnittstelle zur Tourenberechnung
 - Bestehende TMS Core Datenbank-Schnittstelle für die Belieferung der REST-Schnittstelle
 - Bestehende Schnittstellen zum Zurückschreiben der TOP-Service Ergebnisse (berechnete Touren)
- Die Performance sollte durch Hosting von sowohl TMS Datenbanken als auch New Dispo Microservices in GCP auf das Maximum gebracht werden (kein Cross-Cloud Traffic)

Architektur Entscheidungen

- Die grundlegenden architektonischen Entscheidungen zur Fernsteuerung des TMS Core durch New Dispo wurden in einer frühen Phase getätigt
 - Siehe [Decision Log](#)
- Spätere Änderungen wurden in ADRs dokumentiert
 - Siehe [ADR](#)
- Die fortlaufenden Änderungen an TMS Core Integrationen und Erweiterungen wurden in Refinement Contexts (RCs) dokumentiert, Fall-basiert, so wie sie sich zeitlich ergeben haben
 - [RCs](#)

Entwicklung der TMS-Integrationsstrategie über die Zeit

Kern-Bottleneck im TMS Core

- Die Basis-Datenbankobjekte stellten sich über die Zeit als nicht performant heraus, weshalb Ableger dieser von P3 erstellt wurden, um einzelnen Use Cases gerecht zu werden im Sinne der Non-Functional Requirements.
- Das Grundproblem der TMS-Datenbank ist allerdings, dass sehr viele Werte **beim Lesen** aus einzelnen Datenquellen aufgelöst werden und somit die Quelle für langsame Datenzugriffe darstellen (durch verschachtelte und über Datenbank-Funktionen gelöste Mechanik)
- Diesem Grundproblem kann selbst mit dem Erstellen neuer Use-Case-spezifischen Objekte nicht vollends begegnet werden
- Deshalb wurden z.T. Workarounds im New Dispo Stack entwickelt, welche ein stufenweises Zugreifen auf Daten ermöglichen (siehe z.B. das Filtern von Transport Ordnern)

Project G Entfaltung

- In Teilen waren Datenbankfunktionen in PostgreSQL langsamer als in Oracle und führten so zu Performance-Problemen auf unerwartete Weise
 - Diesen Themen kam CAL aber aufgrund der Project G Migrations nicht oder erst sehr spät nach und manche Probleme bestehen bis heute

Tourenberechnung mit TOP-Service und xServer

- Die eigentliche Tourenberechnung findet im xServer statt
- Bis dieser allerdings mit Daten bespielt werden kann, muss vorher eine Schnittstelle in TMS Core durchlaufen werden und anschließend der von CAL entwickelte TOP-Service
 - Auf alle 3 Komponenten hat P3 keinen Einfluss, da sie in der Verantwortung von CAL liegen

Die Summe aller im Laufe des Projekts erstellten Objekte durch P3 sind unten aufgelistet.

Datenbank-Objekte mit "DIS" Präfix

"DIS" wurde als Scope-Präfix für alle Elemente der New Dispo Cloud-Applikation verwendet.

Contributor-Legende: P3 Team | CAL Team

New Dispo Packages (PDIS_*)

Package	Zweck	Erstellt von	Erstellt am	sonjapetkovicP3	Boyan Valchev	mohamadaomar
PDIS_SHIPMENT	Sendungsoperationen, Leg-Erstellung	sonjapetkovicP3	2025-08-26	2	-	-
PDIS_TOURPOINT	Tourpunkt-Management (Zeiten, Intervalle)	sonjapetkovicP3	2025-07-23	7	-	-
PDIS_TRANSPORTORDER	Transport Order CRUD, Fahrzeug-/Anhängierzuzuweisung	JoachimSchreiner	2025-05-21	37	6	2
PDIS_TRANSPORTORDERDTO	DTO Typ-Definitionen	gbing07	2025-02-07	-	-	-

Package	Zweck	Erstellt von	Erstellt am	sonjapetkovicP3	Boyan Valchev	mohamadaomar
PDIS_LEG	Leg-Operationen (Gewicht, Ladestatus)	sonjapetkovicP3	2025-08-25	5	-	-

New Dispo Views (V_DIS_*)

View	Beschreibung	Erstellt von	Erstellt am	sonjapetkovicP3	Boyan Valchev	mohamadomar
V_DIS_TRANSPORTORDER	Kern-View für Transport Orders	gbing07	2025-02-07	4	-	-
V_DIS_TRANSPORTORDER_COUNT	Zählung/Aggregation	gbing07	2025-02-07	-	-	-
V_DIS_TRANSPORTORDER_PICKUPPLANNING	Abholplanung	sonjapetkovicP3	2025-06-16	13	2	1
V_DIS_TRANSPORTORDER_PRESETTEMP	Temperatur-Vorgaben	sonjapetkovicP3	2025-07-15	1	-	-
V_DIS_TRANSPORTORDER_FILTER	Filter-Unterstützung	Jasper Smith	2025-04-29	1	-	-
V_DIS_SHIPMENT	Sendungsdaten	gbing07	2025-02-09	2	-	-
V_DIS_SHIPMENT_ALL	Alle Sendungsdaten	gbing07	2025-02-09	3	-	-
V_DIS_LEG	Leg-Daten	JoachimSchreiner	2025-09-27	-	-	2
V_DIS_TO_TOURPOINT	Tourpunkt-Daten	sonjapetkovicP3	2025-05-16	6	-	3
V_DIS_TO_PRESETTEMP	Temperatur-Vorgaben	JoachimSchreiner	2025-05-21	-	-	-
V_DIS_BRANCH_ADDRESS	Niederlassungsadressen	sonjapetkovicP3	2025-09-08	2	-	-
V_DIS_CONTACT_DETAILS	Kontaktinformationen	gbing07	2024-07-18	-	-	1
V_DIS_FREIGHT_EXCHANGE_TOURPOINTS	Frachtbörse	sonjapetkovicP3	2025-07-15	1	-	-
V_DIS_TOURPOINT_CLIENT_COMMUNICATION	Kundenkommunikation	mohamadaomar	2025-10-17	1	-	1

Zusammenfassung

Kategorie	Anzahl
New Dispo Packages (PDIS_*)	5
New Dispo Views (V_DIS_*)	14
Legacy Dispatcher Packages (DISP_*)	~20
Legacy Dispatcher Tabellen	~24

Alle SQL-Dateien befinden sich in `src/sql/` unter `/view/`, `/package/` und `/table/`.

Infrastrukturelle Situation

- Der New Dispo Stack wird in GCP gehostet und umfasst im Wesentlichen die modernen Komponenten Frontend, Backend, TMS Bridge, TMS Pulse (mit Datastream, Object Store, Cloud Functions und PubSub Bus).
- Der TOP-Service und xServer sind On-Premise Komponenten
- Das Nagel-interne Netzwerk ist mit der GCP-Cloud verbunden.
 - Diese Verbindung liegt außerhalb des Verantwortungsbereichs von P3 bei CAL / Nagel IT

Test-Ergebnisse November 2025

Dispo Test Umgebung Performance:

1. Login in die app dauert ewig (5min +), Thema liegt bei ron, da gibt es grad keine Rückmeldung
2. Laden der Transportaufträge dauert circa 3-4 Sekunden
3. Routenberechnung dauert manchmal 5s + (längste Dauer waren so 30s auch hier: wir wissen, dass es der Call an den top Service ist, der so lange dauert.)
(Dadurch, dass es im Flow beim Hinzufügen von Sendungen die Berechnung getriggert wird, kriegt der User das mit)

Spezifiziertes Feedback zu anderen Performance Themen hat Max K. von Max B. noch nicht bekommen.

Ergo: Schlechte Performance der TMS Legacy Systeme beeinflusst die New Dispo App Performance (User Experience) massiv.

Nicht New Dispo relevante Objekte

Die folgenden Objekte enthalten zwar "DIS" im Namen, sind aber nicht Teil der New Dispo Applikation:

Legacy Dispatcher Objekte (DISP_*)

Zusätzlich existieren Legacy-Objekte mit DISP_ Präfix (nicht New Dispo):

- **Packages (~20):** DISP_MDE_AH , DISP_MDE_B0 , DISP_LESEN , DISP_STATUS , DISP_REORG , etc.
- **Tabellen:** dispo_memory , disp_sort , disp_zuf , v_dispo + ~20 temp Tabellen (tmp*abhdisp)
- **Views:** V_SEN_DISPO_NV* , V_NET_TA_DISP* , V_KVN_TA_LAD_DIS

System-Utilities

Package	Zweck
PDIS_SYS	System-Utilities, Fehlerbehandlung

Distanz-Objekte (DIST/PDIST)

- **Tabellen:** dist , dist_land , dist_vers , distort , distort_land , distort_strasse
- **Packages:** PDIST , PDIST_LIB , PDIST_SESSION